Generación de variables aleatorias: transformada inversa y aceptación–rechazo

Nayeli Cruz Fonseca · Licenciatura en Actuaría · Octubre de 2026

Introducción

Una de las herramientas fundamentales de la **estadística computacional** es la capacidad de simular variables aleatorias que siguen una distribución de probabilidad dada de antemano. La clase parte de una idea sencilla: si puedo elegir números al azar en el intervalo (0, 1), es decir, si tengo una muestra de variables **uniformes**, entonces puedo generar **cualquier** variable aleatoria usando el algoritmo adecuado.

Para lograrlo, la clase presenta **dos métodos**: el muestreo por **transformada inversa** y el método de **aceptación–rechazo**. Los dos dependen de contar con un buen generador de números pseudoaleatorios uniformes, y los ejemplos se trabajan en **Python**.

Lo que yo rescato es que la simulación no es un truco de programación, sino una forma de **expresar la realidad en términos matemáticos** cuando el fenómeno es azaroso. La grabación se corta justo al terminar de explicar el algoritmo de aceptación–rechazo y antes de pasar a los ejemplos en Jupyter, así que completé el resumen con la justificación del método, ejemplos numéricos, código y gráficas propios.

1. Números aleatorios y pseudoaleatorios

Las computadoras **todavía no pueden generar números verdaderamente aleatorios** de forma física. Lo que hacen es usar algoritmos deterministas que producen sucesiones que **se comportan como si fueran aleatorias**: los números **pseudoaleatorios**. En la clase, para fines prácticos, los números aleatorios y los pseudoaleatorios se consideran **lo mismo**.

Los lenguajes orientados a objetos traen su propio generador; en este curso se usa **Python**, cuyo generador es suficiente para lo que se necesita. Para entender la idea, agrego un ejemplo clásico que no viene en la clase, el **generador congruencial lineal**:

$$x_{n+1} = (ax_n + b) \bmod m, \qquad u_{n+1} = \frac{x_{n+1}}{m}$$

En palabras: $siguiente = residuo\ de\ (multiplicador \times anterior + incremento)\ entre\ m;\ \ uniforme = \frac{siguiente}{m}$

Con a = 16807, b = 0, m = $2^{31} - 1$ y semilla $x_0 = 12345$, los primeros tres números son 0.0966, 0.8340 y 0.9477. Parecen azar, pero si repito la semilla obtengo exactamente la misma sucesión. Eso es justo lo que permite **reproducir** una simulación. Python usa hoy un generador más moderno (PCG64), pero la idea es la misma.

1.1 ¿Para qué se necesitan números aleatorios?

La clase da una respuesta que, como dice el profesor, es bastante extensa. La resumo en dos grandes usos:

1. **Modelar fenómenos azarosos.** Por ejemplo, el **movimiento browniano** y la física de partículas en la física estadística; la **dinámica de poblaciones** en procesos estocásticos de evolución; y los **mercados financieros**, donde la simulación **Monte Carlo** ayuda a simular portafolios de inversión y las series de tiempo ayudan a entender su evolución. Para esto no basta con números uniformes: se necesitan números que **provengan de ciertas distribuciones**.

2. **Integración Monte Carlo.** Calcular integrales que no se pueden resolver con los métodos convencionales de cálculo, ni siquiera fácilmente con métodos numéricos, aproximándolas con sucesiones de puntos aleatorios. La clase sólo lo menciona; abajo agrego un ejemplo.

Ejemplo propio de integración Monte Carlo. La integral $\int_0^1 e^{-x^2}\,dx$ no tiene primitiva elemental. Pero es igual a $E[e^{-U^2}]$ con U ~ Uniforme(0, 1), así que la puedo aproximar con un promedio:

$$\int_0^1 e^{-x^2}\,dx \;\approx\; \frac{1}{n}\sum_{i=1}^{n} e^{-u_i{}^2} \qquad con\ u_i\ uniformes\ en\ (0,1)$$

En palabras: integral ≈ promedio de la función evaluada en n puntos elegidos al azar

Con n = 100,000 obtuve 0.7477 con un error estándar de 0.0006; el valor exacto es 0.7468. Por la ley de los grandes números, la estimación se acerca al valor real conforme aumento n.

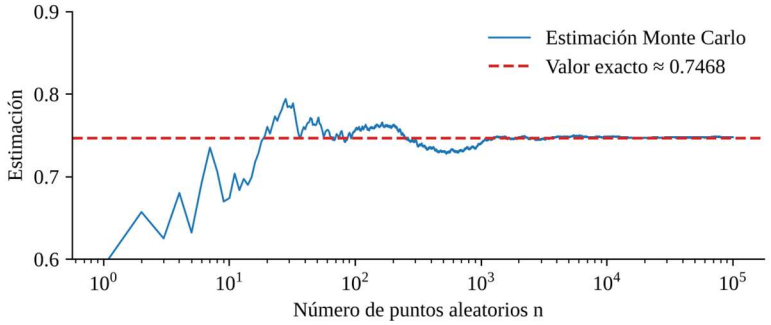


Figura 1. Convergencia de la estimación Monte Carlo de la integral de e^(−x²) en [0, 1] (eje horizontal en escala logarítmica).

1.2 El problema a resolver

El planteamiento de la clase es: **se requiere generar números aleatorios con alguna distribución deseada** a partir de uniformes. Para eso están los dos métodos que siguen.

2. Método de la transformada inversa

2.1 Definición

Sea X una variable aleatoria con función de distribución F_X. Su **transformada inversa** toma un número y entre 0 y 1 y busca, entre todos los números reales donde la función de distribución alcanza o sobrepasa a y, el más pequeño:

$$F_X^{\leftarrow}(y) \;=\; \inf\{x \in \mathbb{R} : F_X(x) \geq y\}, \qquad 0 < y < 1$$

En palabras: transformada inversa en y =
el primer valor de x en el que la probabilidad acumulada llega al nivel y

2.2 La idea clave

El hecho fundamental que subraya la clase es que, **en general, la transformada inversa no es la inversa de la función de distribución**. Sin embargo, en la gran mayoría de las variables aleatorias conocidas sí lo es:

Resultado. Si X tiene función de distribución F **continua y estrictamente creciente** (es decir, una biyección entre $\mathbb{R}$, o el soporte de X, y el intervalo (0, 1)), entonces la transformada inversa es exactamente la función inversa F^{-1}. Además, si U es uniforme en (0, 1):

$$F^{-1}(U) \ \ tiene\ la\ misma\ distribución\ que\ \ X$$

La razón es una cuenta de una línea: $\mathbb{P}(F^{-1}(U) \leq x) = \mathbb{P}(U \leq F(x)) = F(x)$. En mis palabras: **eligiendo números al azar en (0, 1) y aplicándoles la inversa de F, obtengo una muestra con la distribución que quiero**.

2.3 Algoritmo y ejemplo

1. Generar u uniforme en (0, 1).
2. Calcular $x = F^{-1}(u)$.
3. Repetir de forma independiente tantas veces como valores necesite.

Ejemplo (exponencial). Si $F(x) = 1 - e^{-\lambda x}$ para $x \geq 0$, despejando $F(x) = u$ se obtiene

$$F^{-1}(u) = -\frac{1}{\lambda} ln(1 - u)$$

Con $\lambda = 2$ y $u = 0.7$ resulta $x = -(1/2) \ln(0.3) \approx 0.6020$. Una limitación que yo noto: el método necesita poder **despejar** la inversa de F. Para distribuciones como la normal o la beta no hay una fórmula cerrada, y ahí es donde entra el segundo método.

3. Método de aceptación–rechazo

3.1 Planteamiento

Quiero generar una variable X con densidad f, pero no sé simularla directamente. La idea de la clase es buscar una **distribución alternativa** con densidad g que sí sepa simular de forma eficiente (por ejemplo, con la transformada inversa) y usarla como "propuesta".

Hipótesis importante. El cociente de la densidad original entre la alternativa debe estar **acotado por una constante positiva** c en todos los puntos donde están definidas:

$$\frac{f(x)}{g(x)} \leq c \ \ para\ todo\ x, \qquad es\ decir,\ \ f(x) \leq c \cdot g(x)$$

En palabras: $\frac{densidad\ que\ quiero}{densidad\ que\ sé\ simular} \leq constante \iff$ *la curva* $c \cdot g$ *queda siempre por encima de* f

3.2 El algoritmo

Si se cumple la condición, el método dice:

1. Generar Y con densidad g (por ejemplo, con la transformada inversa o algún otro método).
2. Generar U uniforme en (0, 1), **independiente** de Y.
3. Si $U \leq f(Y) / (c \cdot g(Y))$, **aceptar** y tomar $X = Y$.
4. En otro caso, **rechazar** y volver al paso 1, hasta que se acepte un valor.

$$Aceptar\ \ Y\ \ si\ \ U \leq \frac{f(Y)}{c \cdot g(Y)}$$

En palabras: acepto la propuesta si el número uniforme queda por debajo de $\frac{densidad\ deseada\ en\ Y}{(constante)(densidad\ de\ propuesta\ en\ Y)}$

Lo interpreto así: el cociente $f(Y)/(c \cdot g(Y))$ está entre 0 y 1 y mide **qué tan probable es que la propuesta Y venga de f**. Donde f es alta comparada con $c \cdot g$, casi siempre acepto; donde f es baja, casi siempre rechazo. Así voy "recortando" la muestra de g hasta que tenga la forma de f.

3.3 ¿Por qué funciona?

La grabación se corta antes de la justificación, así que la agrego. La probabilidad de aceptar en un intento es

$$\mathbb{P}(aceptar) \;=\; \int \frac{f(y)}{c\,g(y)}\, g(y)\, dy \;=\; \frac{1}{c}\int f(y)\, dy \;=\; \frac{1}{c}$$

y la probabilidad de aceptar un valor menor o igual que x es $\int_{-\infty}^{x} \frac{f(y)}{c}\, dy \;=\; F(x)/c$. Por lo tanto, la distribución de los valores aceptados es

$$\mathbb{P}(Y \;\le\; x \;\mid\; aceptar) \;=\; \frac{F(x)\,/\,c}{1\,/\,c} \;=\; F(x)$$

Es decir, **los valores aceptados tienen exactamente la densidad f**. Además, cada intento es independiente y se acepta con probabilidad 1/c, así que el número de intentos hasta aceptar es **geométrico con media c**. Por eso conviene escoger la constante lo más pequeña posible:

$$c \;=\; sup_x \frac{f(x)}{g(x)}, \qquad \mathbb{P}(aceptar) \;=\; \frac{1}{c}, \qquad E[número\ de\ intentos] \;=\; c$$

3.4 Ejemplo propio: una distribución Beta(2, 2)

Quiero simular X con densidad f(x) = 6x(1 − x) en [0, 1], que no tiene una inversa fácil de despejar. Uso como propuesta la uniforme, g(x) = 1 en [0, 1]. El máximo de f está en x = 1/2:

$$c \;=\; máx_{0\le x\le 1}\, 6x(1 \;-\; x) \;=\; 6 \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{2}, \qquad \mathbb{P}(aceptar) \;=\; \frac{2}{3}, \qquad \frac{f(y)}{c\,g(y)} \;=\; 4y(1 \;-\; y)$$

Por ejemplo, si Y = 0.3 entonces f(Y)/(c·g(Y)) = 4(0.3)(0.7) = 0.84: acepto si U ≤ 0.84. Si Y = 0.05, el cociente es 0.19 y lo más probable es rechazar, porque f es muy baja cerca de los extremos.

```
import numpy as np
rng = np.random.default_rng(2026)

f = lambda x: 6 * x * (1 - x)   # densidad que quiero (Beta(2, 2))
c = 1.5                          # cota de f/g con g uniforme

def acepta_rechaza():
    while True:
        y = rng.uniform()        # 1. propuesta con densidad g
        u = rng.uniform()        # 2. uniforme independiente
        if u <= f(y) / c:        # 3. criterio de aceptación
            return y             #    se acepta: X = Y
                                 # 4. si no, se repite

muestra = np.array([acepta_rechaza() for _ in range(10_000)])
print(muestra.mean(), muestra.var())   # teóricos: 1/2 y 1/20
```

Al correrlo obtuve una **tasa de aceptación de 0.671** (teórica 2/3), un promedio de **1.50 intentos** por valor (teórico c = 3/2), una **media de 0.501** (teórica 1/2) y una **varianza de 0.0509** (teórica 1/20 = 0.05).

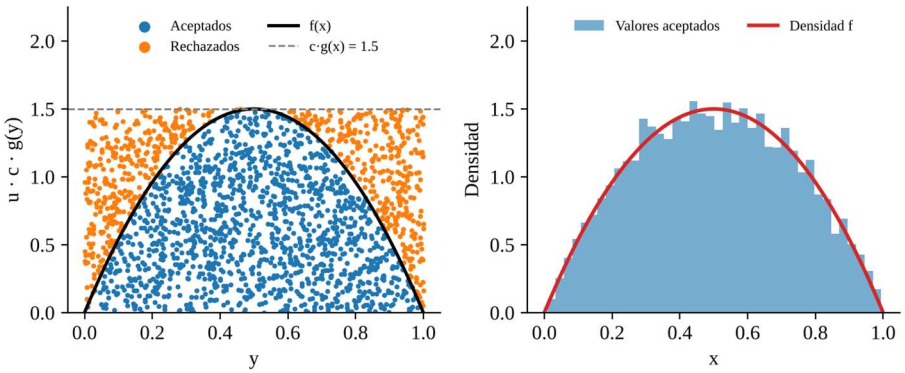


Figura 2. Izquierda: puntos propuestos; se aceptan los que quedan debajo de f. Derecha: histograma de los valores aceptados contra la densidad Beta(2, 2).

En mis palabras: la gráfica de la izquierda explica todo el método. Los puntos se reparten de manera uniforme en el rectángulo de altura c·g = 1.5, y sólo me quedo con los que caen **debajo de la curva f**. Por eso los aceptados terminan teniendo la forma de f, como se ve a la derecha.

4. Comparación de los dos métodos

	Transformada inversa	**Aceptación–rechazo**
Qué necesita	Poder calcular F^{-1} (o F← en general)	Una densidad g fácil de simular y una cota $f \le c \cdot g$
Uniformes por valor	Exactamente una	En promedio 2c (dos por intento)
Desperdicio	Ninguno	Se rechaza una fracción $1 - 1/c$
Ventaja	Muy rápido y exacto	Sirve aunque F no se pueda invertir
Ejemplos	Exponencial, uniforme, discretas	Beta, normal (con propuesta exponencial o de Cauchy)

Los dos métodos se complementan: la transformada inversa muchas veces es la que genera la propuesta g dentro del método de aceptación–rechazo, tal como menciona la clase.

Conclusión

Después de ver la clase, me queda claro que todo lo que se simula en estadística computacional descansa en una base muy simple: **números pseudoaleatorios uniformes en (0, 1)**. A partir de ellos, la transformada inversa genera directamente cualquier variable cuya función de distribución se pueda invertir, porque $F^{-1}(U)$ tiene la misma distribución que X cuando F es continua y estrictamente creciente.

Cuando no es posible invertir F, el método de aceptación–rechazo propone valores de una densidad más sencilla g y se queda sólo con los que quedan debajo de f, siempre que $f \le c \cdot g$. Cada valor aceptado tiene exactamente la distribución deseada y, en promedio, se necesitan c intentos, por lo que conviene que c sea lo más pequeña posible.

Lo más valioso para mí es ver por qué estos métodos importan en actuaría y finanzas: son la base de la **simulación Monte Carlo**, que permite simular portafolios, siniestros o trayectorias de procesos estocásticos y calcular integrales que no se pueden resolver a mano.